



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

**Кафедра инструментального и прикладного программного обеспечения
(ИиППО)**

КУРСОВАЯ РАБОТА

по дисциплине: Проектирование и разработка клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: «Интерактивный конструктор образовательных тестов»

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Работа представлена к защите _____ (дата) _____ /Миркин К.Л./
(подпись и ф.и.о. студента)

Руководитель: ст. преподаватель Сеницын Анатолий Васильевич

Работа допущена к защите _____ (дата) _____ /Сеницын А.В./
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____ / ст. преподаватель Сеницын А.В. /
_____ / _____ /

М. РТУ МИРЭА. 2026



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Проектирование и разработка клиент-серверных приложений
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Срок представления к защите: 18.05.2026

Руководитель: ст. преп. Сеницын Анатолий Васильевич

Тема: «Интерактивный конструктор образовательных тестов»

Исходные данные: методология двенадцати факторов, Git, Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области для выбранной темы. 2. Выбрать клиент-серверную архитектуру для разрабатываемого приложения и дать её детальное описание с помощью UML. 3. Выбрать программный стек для реализации фуллстек CRUD приложения. 4. Разработать клиентскую и серверную части приложения, реализовать авторизацию и аутентификацию пользователя, обеспечить работу с базой данных, заполнить тестовыми данными, валидировать ролевою модель на некорректные данные. 5. Провести фаззинг-тестирование. 6. Разместить исходный код клиент-серверного приложения в системе контроля версий с наличием Dockerfile и описанием структуры проекта в readme файле. 7. Развернуть клиент-серверное приложение в облаке. 8. Разработать презентацию с графическими материалами.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: Болбаков Р. Г. / Болбаков Р. Г. /, «18» февраля 2026 г.

Задание на КР выдал: Сеницын А. В. / Сеницын А. В. /, «18» февраля 2026 г.

Задание на КР получил: Миркин К. Л. / Миркин К. Л. /, «18» февраля 2026 г.

РЕФЕРАТ

Отчёт 47 с., 10 рис., 4 табл., 22 ист.

FASTAPI, REST API, МУЛЬТИМЕДИЙНЫЙ КОНСТРУКТОР ТЕСТОВ

Миркин К.Л., Курсовая работа направления подготовки «Программная инженерия» предмета «Бэкенд-разработка» на тему «Серверная часть веб-приложения — интерактивного конструктора образовательных тестов»: М. 2026 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО)

Курсовая работа посвящена разработке серверной части веб-приложения — интерактивного конструктора образовательных тестов. Выполнен анализ предметной области систем автоматизированного тестирования, сформированы функциональные и нефункциональные требования, проведён обзор существующих решений (Google Forms, Online Test Pad, Quizizz, Moodle) с построением матриц функционального и архитектурного сравнения.

Спроектирована клиент-серверная архитектура системы: клиентская часть — одностраничное приложение по методологии Feature-Sliced Design, серверная — многослойная архитектура на базе FastAPI. Описана модель данных (реляционная схема в третьей нормальной форме) и модель развёртывания на контейнеризованной инфраструктуре.

Реализована серверная часть на языке Python с использованием фреймворка FastAPI, СУБД PostgreSQL, объектного хранилища MinIO, а также клиентская часть на React с TypeScript. Приведены листинги кода, демонстрирующие применение многослойной архитектуры, паттерна Repository и внедрения зависимостей. Корректность реализации подтверждена модульными, интеграционными и фаззинг-тестами.

СОДЕРЖАНИЕ

Определения, обозначения и сокращения	7
Введение	11
1 Анализ предметной области и требований	13
1.1 Актуальность и новизна	13
1.2 Описание предметной области	13
1.3 Проблемы предметной области	14
1.4 Анализ существующих решений	15
1.4.1 Google Forms	15
1.4.2 Online Test Pad	16
1.4.3 Quizizz	17
1.4.4 Moodle (модуль Quiz)	18
1.4.5 Матрица сравнения аналогов	19
1.5 Требования к системе	20
1.5.1 Функциональные требования	20
1.5.2 Нефункциональные требования	21
1.6 Итоги анализа предметной области	22
2 Проектирование архитектуры системы	24
2.1 Общая архитектура системы	24
2.1.1 Архитектура клиентской части	24
2.1.2 Архитектура серверной части	26
2.1.3 Модель данных	28
2.1.4 Модель развёртывания	30
2.2 Выбор технологического стека	32
2.2.1 Клиентская часть	32
2.2.2 Серверная часть	33
2.2.3 Хранение данных	35
2.2.4 Инфраструктура	35
2.3 Итоги	36
3 Разработка, тестирование и развертывание системы	37
3.1 Разработка серверной части	37
3.2 Разработка клиентской части	38

3.3 Тестирование системы	39
3.3.1 Модульное и интеграционное тестирование	39
3.3.2 Фаззинг тестирование	39
3.4 Развёртывание системы	41
3.4.1 Контейнеризация серверной части	41
3.4.2 Контейнеризация клиентской части	41
3.4.3 Оркестрация production-среды	41
3.5 Итоги	42
Заключение	44
Список использованных источников	46

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчёте применяют следующие определения, обозначения и сокращения.

ACID-транзакции (Atomicity, Consistency, Isolation, Durability)	– набор свойств транзакций базы данных, гарантирующих атомарность, согласованность, изолированность и долговечность операций.
Alembic	– инструмент управления миграциями схемы базы данных для SQLAlchemy, обеспечивающий версионный контроль изменений структуры данных.
API (Application Programming Interface)	– программный интерфейс, определяющий способы взаимодействия компонентов программного обеспечения.
AsyncSession	– асинхронная сессия SQLAlchemy для выполнения запросов к базе данных в режиме asyncio без блокировки event loop.
Clean Architecture	– архитектурный паттерн, предложенный Робертом Мартином, обеспечивающий независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов.
Dependency Injection (DI)	– паттерн проектирования, при котором зависимости компонентов передаются извне, а не создаются внутри компонента.
Docker Compose	– инструмент оркестрации многоконтейнерных Docker-приложений, позволяющий декларативно описывать сервисы, сети и тома.
DTO (Data Transfer Object)	– объект передачи данных, используемый для обмена структурированной информацией между слоями приложения.
ER-диаграмма (Entity-Relationship Diagram)	– диаграмма «сущность–связь», используемая для моделирования структуры данных и отношений между сущностями.

FastAPI	– современный веб-фреймворк для Python, ориентированный на создание REST API с поддержкой асинхронности, автоматической валидации данных и генерации документации OpenAPI.
Feature-Sliced Design (FSD)	– методология организации фронтенд-приложений, основанная на послойной декомпозиции с однонаправленными зависимостями.
HTTP (HyperText Transfer Protocol)	– протокол передачи данных между клиентом и сервером в веб-среде.
Hypothesis	– библиотека property-based тестирования для Python, позволяющая генерировать произвольные входные данные для проверки инвариантов программы.
JWT (JSON Web Token)	– компактный и самодостаточный формат для передачи данных в виде JSON-объекта, используемый для безсессионной аутентификации.
MinIO	– S3-совместимое объектное хранилище с открытым исходным кодом, разворачиваемое как Docker-контейнер.
OpenAPI	– спецификация описания REST API, автоматически генерируемая FastAPI из аннотированных Python-функций.
ORM (Object-Relational Mapping)	– технология отображения объектных структур в реляционные базы данных.
Presigned URL	– подписанный URL для временного прямого доступа к объекту в S3-совместимом хранилище без прохождения через сервер приложений.
Pydantic	– библиотека для декларативной валидации данных в Python на основе аннотаций типов.

Repository	– паттерн, инкапсулирующий логику доступа к данным и предоставляющий интерфейс для операций с сущностями.
REST (Representational State Transfer)	– архитектурный стиль взаимодействия распределённых систем, использующий принципы унифицированного интерфейса и обмена представлениями ресурсов.
S3 (Simple Storage Service)	– протокол объектного хранения данных, разработанный Amazon Web Services и ставший отраслевым стандартом.
S3-совместимое API	– интерфейс объектного хранилища, реализующий протокол Amazon S3 для работы с бакетами и объектами.
Service Layer	– слой бизнес-логики приложения, обрабатывающий прикладные операции и обращающийся к репозиториям.
SQL (Structured Query Language)	– язык запросов, используемый для управления реляционными базами данных.
SQLAlchemy	– библиотека ORM и инструмент работы с реляционными базами данных в Python.
TanStack Query	– библиотека для управления серверным состоянием в React-приложениях, обеспечивающая кеширование, фоновое обновление и дедупликацию запросов.
Typst (typst.ts)	– язык вёрстки документов с поддержкой математических формул и форматированного текста; в проекте используется WASM-компилятор typst.ts для рендеринга в браузере.
Uvicorn	– асинхронный сервер приложений ASGI для Python, используемый для запуска FastAPI.

Vite	– инструмент сборки фронтенд-приложений, обеспечивающий быстрый HMR в разработке и оптимизированный бандл в продакшне.
Zustand	– минималистичная библиотека управления локальным состоянием для React без необходимости провайдеров контекста.
СУБД	– система управления базами данных.
Фаззинг-тестирование (fuzzing)	– метод тестирования программного обеспечения с использованием псевдослучайных или специально сформированных входных данных для выявления ошибок и уязвимостей.
Фреймворк	– программный каркас, упрощающий разработку приложений и задающий архитектурные принципы.

ВВЕДЕНИЕ

Цифровизация образования в России приобретает устойчивый характер: число обучающихся в онлайн-формате выросло с 3,1 млн человек в 2019 году до 8,1 млн к 2024–2025 учебному году [1]. В этих условиях педагогам необходимо постоянно расширять цифровые компетенции и осваивать новые инструменты учебной деятельности, среди которых особое место занимают интерактивные образовательные тесты [2]. Автоматически проверяемые тесты позволяют как отслеживать текущий прогресс обучающихся (формативное оценивание), так и проводить итоговый контроль знаний, оперативно выявляя пробелы без дополнительных трудозатрат преподавателя [3].

Вместе с тем существующие инструменты создания тестов в полной мере не отвечают потребностям современного преподавателя: они либо не поддерживают математические формулы и мультимедийный контент непосредственно в теле вопроса, либо представляют собой закрытые платформы с хранением данных на серверах зарубежных компаний. Глобальный рынок онлайн-платформ для тестирования в 2024 году оценивается в 3,18 млрд долларов США; по прогнозам, к 2032 году он достигнет 10,14 млрд при среднегодовом темпе роста 15,6 % [4]. Высокий темп роста сегмента отражает устойчивый спрос как в академической среде, так и в корпоративном обучении.

Актуальность выбранной темы определяется необходимостью разработки серверной части специализированного инструмента для создания, проведения и автоматической проверки образовательных тестов на основе современных архитектурных паттернов и технологического стека. Особое внимание уделяется применению многослойной архитектуры и принципов чистой архитектуры [5], обеспечивающих независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов.

Цель работы – разработка серверной части веб-приложения — интерактивного конструктора образовательных тестов с поддержкой математических формул, мультимедийного контента и гибкого управления доступом.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области автоматизированного тестирования и сформулировать требования к системе;
- провести обзор и сравнительный анализ существующих инструментов создания тестов (Google Forms, Online Test Pad, Quizizz, Moodle);
- спроектировать архитектуру клиент-серверной системы, включая модель данных и модель развёртывания;
- обосновать выбор технологического стека (FastAPI, PostgreSQL, SQLAlchemy, React, TypeScript, Docker);
- реализовать серверную часть приложения, включая REST API, слой бизнес-логики и слой доступа к данным;
- реализовать клиентскую часть приложения с поддержкой рендеринга формул и медиаконтента;
- выполнить тестирование системы, включая фаззинг-тестирование API;
- обеспечить контейнеризованное развёртывание системы.

Объект исследования – процессы создания, проведения и автоматической проверки образовательных тестов в цифровой среде.

Предмет исследования – архитектурные и технологические решения при разработке серверной части интерактивного конструктора образовательных тестов с применением многослойной архитектуры и современных веб-технологий.

1 Анализ предметной области и требований

1.1 Актуальность и новизна

Цифровизация образования в России приобретает устойчивый характер: по данным аналитического обзора платформ дистанционного обучения, число обучающихся в онлайн-формате выросло с 3,1 млн человек в 2019 году до 8,1 млн к 2024–2025 учебному году [1]. В этих условиях педагогам необходимо постоянно расширять цифровые компетенции и осваивать новые инструменты учебной деятельности, среди которых особое место занимают интерактивные образовательные тесты [2]. Автоматически проверяемые тесты позволяют как отслеживать текущий прогресс обучающихся (формативное оценивание), так и проводить итоговый контроль знаний, оперативно выявляя пробелы без дополнительных трудозатрат преподавателя [3].

Глобальный рынок онлайн-платформ для тестирования в 2024 году оценивается в 3,18 млрд долларов США; по прогнозам, к 2032 году он достигнет 10,14 млрд при среднегодовом темпе роста 15,6 % [4]. Высокий темп роста сегмента отражает устойчивый спрос как в академической среде, так и в корпоративном обучении.

Вместе с тем существующие инструменты создания тестов в полной мере не отвечают потребностям современного преподавателя: они либо не поддерживают математические формулы и мультимедийный контент непосредственно в теле вопроса, либо представляют собой закрытые платформы с хранением данных на серверах зарубежных компаний. Разрабатываемая система ориентирована на устранение этих ограничений: она предоставляет мультимедийный конструктор с поддержкой форматирования и математических выражений, гибким управлением доступом к тестам и возможностью развёртывания на собственной инфраструктуре.

1.2 Описание предметной области

Образовательный тест представляет собой структурированный набор заданий, предназначенных для проверки усвоения знаний обучающимися. В педагогической практике тестирование применяется как для текущего контроля успеваемости, так и для итоговой аттестации. Автором теста, как

правило, выступает преподаватель или методист; тестируемыми — студенты, школьники или иные обучающиеся.

Сложившийся процесс создания и проведения тестирования в типичной образовательной организации выглядит следующим образом:

- Преподаватель формирует тест вручную в текстовом редакторе; математические формулы оформляются как встроенные изображения или текстовые суррогаты.

- Готовый тест распространяется по электронной почте, через мессенджер или в виде ссылки на универсальную веб-форму, не предназначенную специально для тестирования.

- Медиамаериалы – рисунки, аудиозаписи, видеофрагменты – передаются отдельными файлами и не встроены в тело задания.

- Проверка ответов на вопросы со свободным вводом выполняется вручную.

- Результаты фиксируются в электронной таблице; агрегированной статистики по отдельному тесту нет.

- Тесты хранятся как разрозненные файлы: единого репозитория с поиском и повторным использованием материалов у педагога нет.

1.3 Проблемы предметной области

На основе описанного процесса выявлены следующие проблемы, обуславливающие необходимость разработки специализированного инструмента.

- 1) Отсутствует единое цифровое пространство для хранения, поиска и повторного использования образовательных тестов.

- 2) Существующие бесплатные инструменты не поддерживают нативное отображение математических формул и лёгкое форматирование непосредственно в тексте вопроса, что особенно критично для точных и естественных наук.

- 3) Медиаконтент — изображения, аудиозаписи и видеофрагменты — не интегрируется в тело вопроса, а передаётся отдельно, нарушая целостность задания.

4) Отсутствует гибкое управление доступом: нет возможности одновременно поддерживать как публичные тесты в общем каталоге, так и приватные, доступные исключительно по ссылке.

5) Вопросы со свободным вводом ответа либо не поддерживаются, либо требуют ручной проверки, что снижает ценность автоматизированного тестирования.

6) Широко распространённые платформы хранят данные пользователей и тесты на серверах зарубежных компаний, что создаёт правовые и санкционные риски для российских образовательных организаций.

7) Доступные бесплатные решения имеют устаревший интерфейс или существенно ограниченный функционал, а коммерческие платформы требуют значительных финансовых вложений.

1.4 Анализ существующих решений

Для обоснования требований к разрабатываемой системе проведён анализ четырёх широко используемых инструментов создания и проведения образовательных тестов.

1.4.1 Google Forms

Google Forms — универсальный конструктор форм от Google, применяемый в том числе для создания тестов. Поддерживает вопросы с выбором ответа, коротким и произвольным вводом. При включении режима теста автоматически проверяются ответы и выставляются баллы; результаты экспортируются в Google Sheets. Инструмент полностью бесплатен, не имеет лимита участников и интегрирован с экосистемой Google.

При этом для образовательного тестирования возможностей Google Forms недостаточно. Математические формулы в тексте вопроса не поддерживаются; изображения допускается вставить лишь в варианты ответа, но не в условие задания; аудио и видео в вопросах отсутствуют полностью. Единого каталога для публикации и поиска тестов не предусмотрено. Данные пользователей и содержимое форм хранятся на серверах Google (США), что создаёт санкционные риски для российских образовательных учреждений.

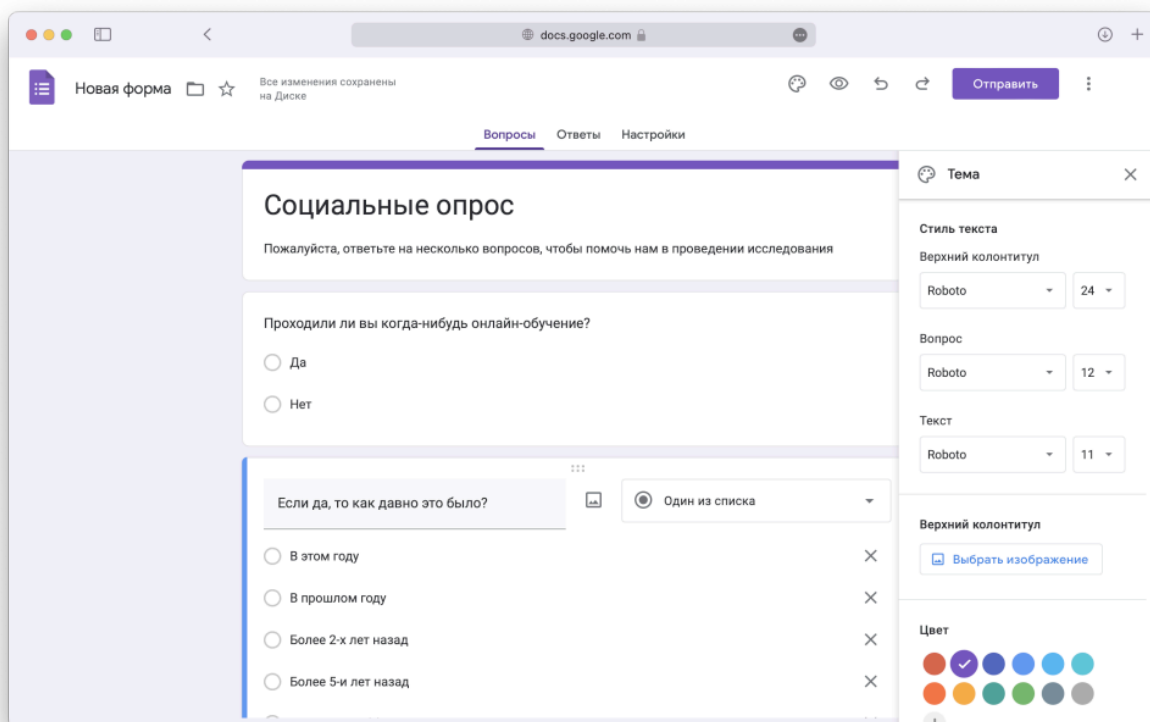


Рисунок 1 – Интерфейс конструктор вопросов в Google Forms

1.4.2 Online Test Pad

Online Test Pad — российский бесплатный онлайн-сервис для создания тестов, опросов, кроссвордов и логических игр [6]. Инструмент не требует установки; участники могут проходить тест без регистрации. Среди бесплатных отечественных решений он рекомендован педагогам как доступный инструмент для разработки интерактивных учебных материалов [2].

Вместе с тем сервис имеет устаревший интерфейс, не соответствующий современным стандартам удобства. Видео и аудио в тексте вопроса не поддерживаются; математические формулы отсутствуют. Публичного каталога с поиском по тегам нет; self-hosted развёртывание и хранение данных на собственном сервере недоступны.

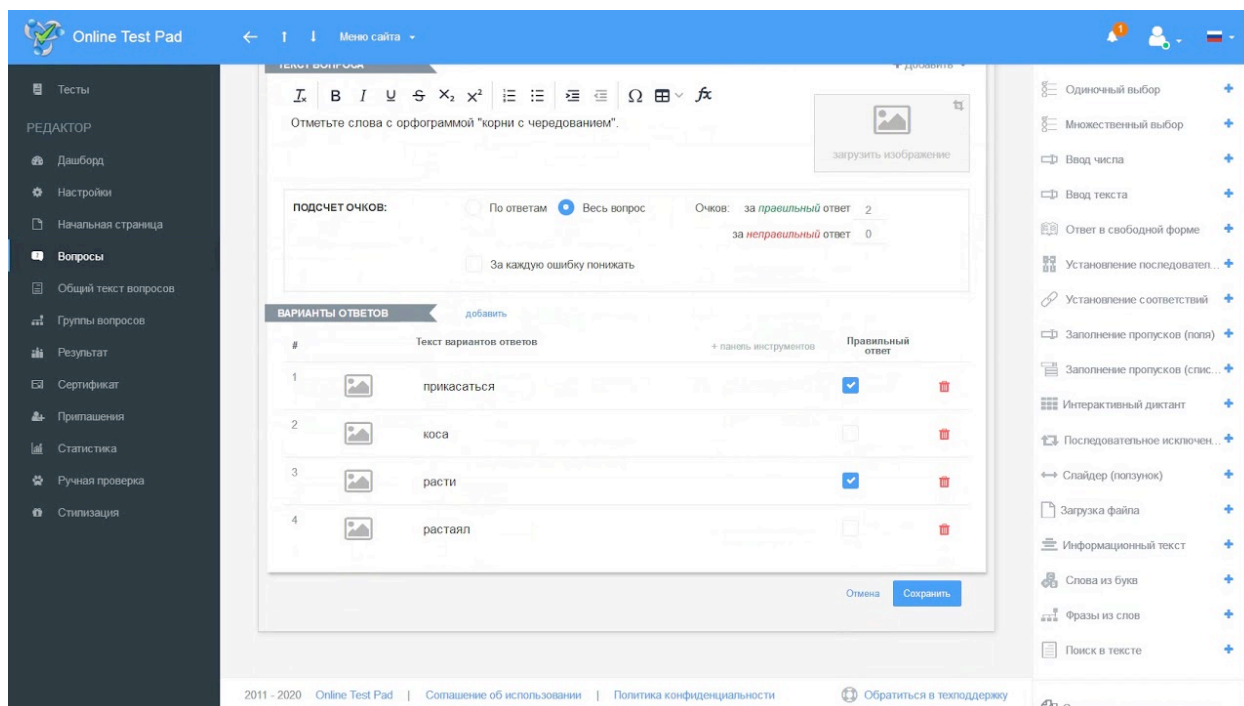


Рисунок 2 – Интерфейс конструктора теста в Online Test Pad

1.4.3 Quizizz

Quizizz — международная асинхронная платформа тестирования: каждый участник проходит задания в собственном темпе. Платформа предоставляет публичную библиотеку из более чем 30 млн материалов [7], инструменты аналитики и функции на основе искусственного интеллекта.

Наличие публичного каталога с поиском — сильная сторона платформы в сравнении с аналогами. Однако поддержка математических формул отсутствует; прикрепление аудио и видео к вопросам доступно лишь в платной подписке. Self-hosted развёртывание невозможно: данные хранятся на серверах Quizizz Inc. (США).

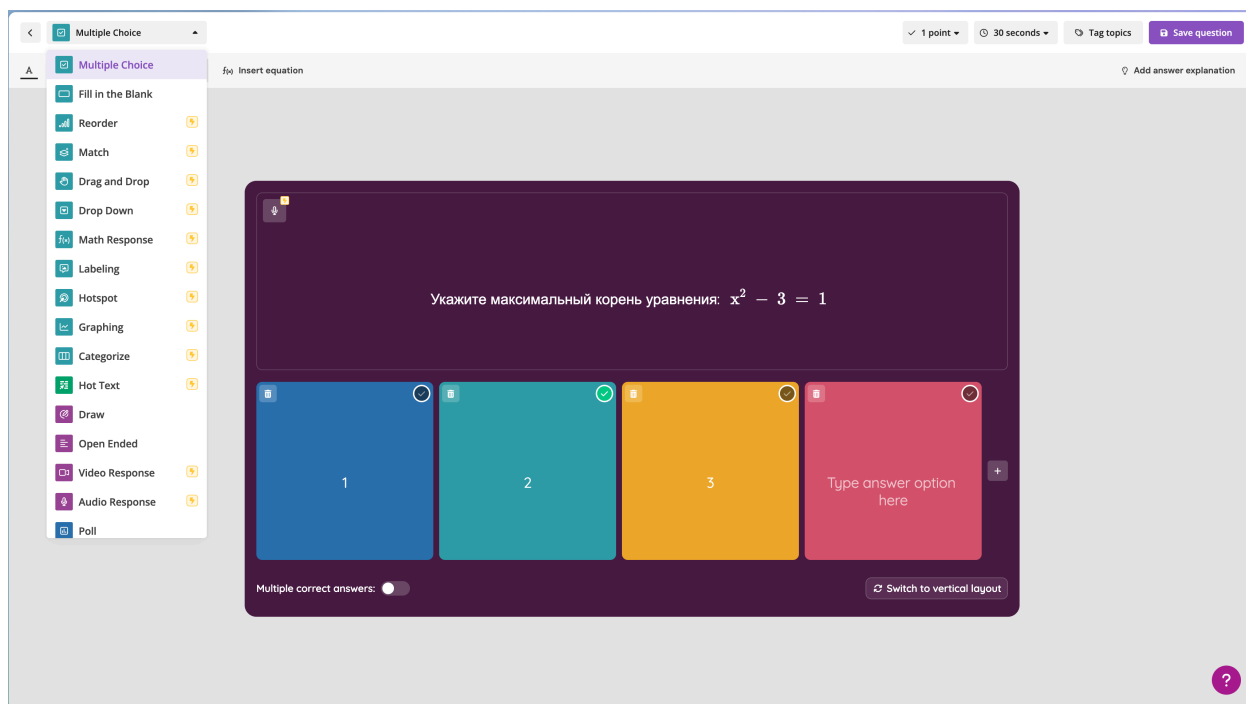


Рисунок 3 – Интерфейс конструктора вопросов в Quizizz

1.4.4 Moodle (модуль Quiz)

Moodle — открытая система управления обучением, широко применяемая в российских университетах. Встроенный модуль Quiz поддерживает около 15 типов вопросов, отображение LaTeX-формул через встроенный фильтр и настройку параметров попыток и времени. По данным библиометрического анализа публикаций в базе Scopus за 2017–2022 годы, Россия занимает первое место по числу научных работ, посвящённых Moodle, — 151 из 1045 зафиксированных документов [8].

Система обеспечивает открытый код, self-hosted развёртывание и хранение данных на собственном сервере — что является её очевидным преимуществом. Однако Moodle разработана как полноценная учебная платформа, а не специализированный конструктор тестов: создание одного теста требует прохождения большого числа экранов настройки, что создаёт высокий порог вхождения для педагогов. Единый публичный каталог тестов с тегами и поиском не предусмотрен. Встроенного хостинга для аудио и видео непосредственно в тексте вопроса нет. Развёртывание требует значительных серверных ресурсов.

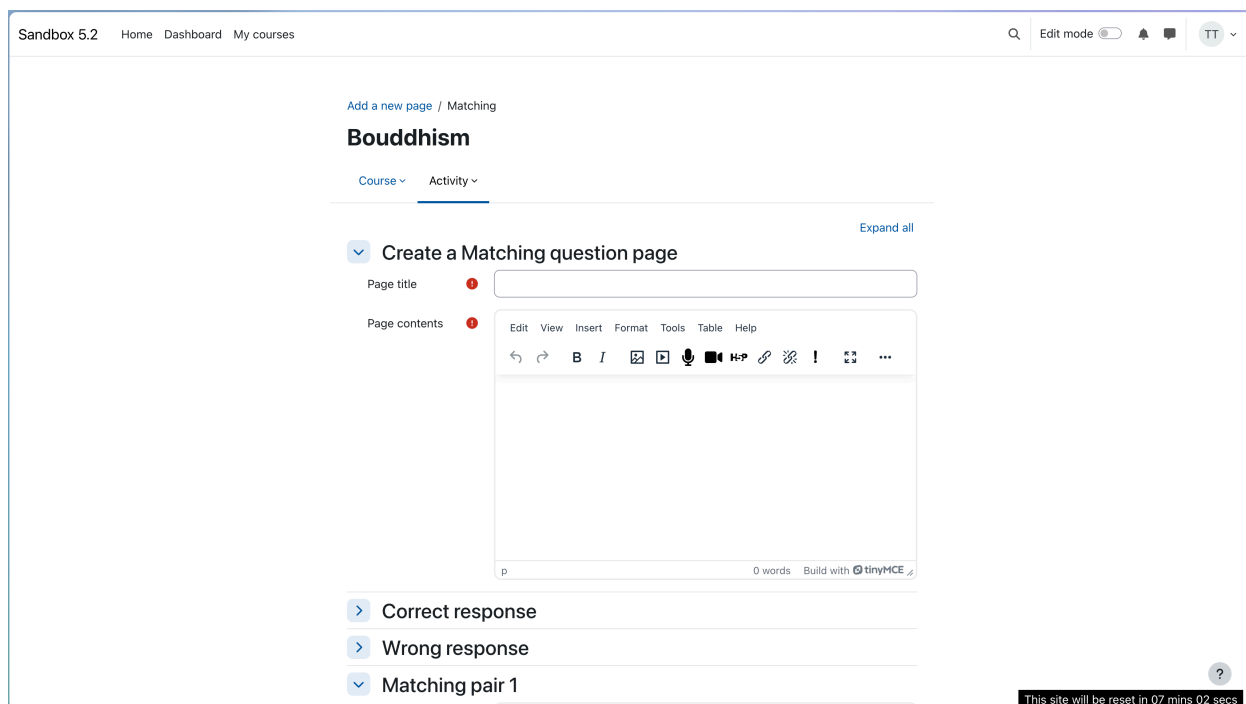


Рисунок 4 – Интерфейс Moodle: конструктор модуля Quiz

1.4.5 Матрица сравнения аналогов

В таблице 1 приведено сравнение рассмотренных инструментов по функциональным возможностям, значимым для образовательного тестирования.

Таблица 1 – Функциональное сравнение аналогов

Возможность	Google Forms	Online Test Pad	Quizizz	Moodle
Вопросы с выбором ответа	Да	Да	Да	Да
Вопросы со свободным вводом	Да	Да	Да	Да
Математические формулы	Нет	Нет	Нет	Да
Изображения в вопросе	Нет	Да	Да	Да
Аудио в вопросе	Нет	Нет	Платно	Да
Видео в вопросе	Нет	Нет	Платно	Да
Публичный каталог с тегами	Нет	Нет	Да	Нет
Приватный доступ по ссылке	Нет	Да	Да	Нет
Автоматическая проверка	Да	Да	Да	Да
Аналитика прохождений	Частично	Да	Да	Да

Таблица 2 отражает архитектурные характеристики, определяющие применимость инструментов в российских образовательных организациях.

Таблица 2 – Архитектурное сравнение аналогов

Характеристика	Google Forms	Online Test Pad	Quizizz	Moodle
Открытый исходный код	Нет	Нет	Нет	Да
Self-hosted развёртывание	Нет	Нет	Нет	Да
Данные на собственном сервере	Нет	Нет	Нет	Да
Бесплатность без ограничений	Да	Да	Частично	Да

Анализ таблиц 1 и 2 показывает, что ни один из рассмотренных инструментов не обеспечивает одновременно: поддержку математических формул, полноценный мультимедийный контент в вопросах, публичный каталог с тегами, гибкое управление доступом и self-hosted развёртывание. Moodle покрывает большинство архитектурных требований, однако уступает в простоте создания тестов и не предоставляет публичного каталога. Это определяет нишу разрабатываемой системы.

1.5 Требования к системе

1.5.1 Функциональные требования

На основании выявленных проблем сформулированы функциональные требования к системе, представленные в таблице 3.

Таблица 3 – Функциональные требования к системе

Группа	Содержание требования
Аутентификация и профиль	Регистрация нового пользователя; вход по логину и паролю; редактирование профиля.
Создание и управление тестами	Создание, редактирование и удаление теста; задание названия, описания и набора тегов; выбор режима видимости: публикация в общем каталоге или доступ по приватной ссылке.
Конструктор вопросов	Добавление, редактирование и удаление вопросов; текст с поддержкой форматирования и математических формул; тип ответа: с вариантами (один или несколько правильных) или со свободным вводом; прикрепление к вопросу медиафайлов — изображения, аудиозаписи, видеофрагмента.
Хранение медиафайлов	Загрузка и хранение медиафайлов в объектном хранилище; ограничение допустимых типов и максимального размера файла.
Поиск и каталог	Полнотекстовый поиск по названию теста; фильтрация по тегам; просмотр публичного каталога.
Прохождение теста	Последовательный показ вопросов с корректным отображением формул и медиаконтента; опциональный таймер.
Автопроверка и результаты	Автоматическая проверка ответов с вариантами; нормализованное сравнение для свободного ввода; расчёт итогового балла; постраничный разбор ответов.
Статистика автора	Количество прохождений и средний балл по созданному тесту.

1.5.2 Нефункциональные требования

Нефункциональные требования определяют эксплуатационные и инфраструктурные характеристики системы и приведены в таблице 4.

Таблица 4 – Нефункциональные требования к системе

Критерий	Метрика	Значение	Обоснование
Производительность	Время ответа (P95)	≤ 100 мс	Отклик до 100 мс воспринимается пользователем как мгновенный и обеспечивает плавную навигацию в конструкторе.
Производительность	Загрузка страницы (FCP)	≤ 3 с	Обеспечивает комфортную первичную загрузку интерфейса на типичном канале связи.
Доступность	Uptime	$\geq 99,9$ %	Бесперебойный доступ в учебные часы и в периоды контрольных мероприятий.
Безопасность	Хранение паролей	Хеширование	Пароли не хранятся в открытом виде; применяется стойкий алгоритм хеширования.
Безопасность	Доступ к приватным тестам	Только по ссылке	Тест недоступен через каталог или прямой перебор идентификаторов.
Безопасность	Загружаемые файлы	Проверка по MIME-типу и размеру	Снижает риск загрузки вредоносного контента.
Совместимость	Браузеры	Chrome, Firefox, Edge, Safari	Охват актуальных браузеров в студенческой и преподавательской среде.
Масштабируемость	Развёртывание	Контейнеризация	Контейнеризованная инфраструктура допускает горизонтальное масштабирование без изменения кода приложения.

1.6 Итоги анализа предметной области

Проведён анализ предметной области конструкторов образовательных тестов. Описан сложившийся процесс создания и проведения тестирования, на основании которого выявлены системные проблемы: отсутствие единого репозитория тестов, ограниченная поддержка мультимедиа и математических формул, трудности с управлением доступом, риски, связанные с хранением данных на серверах зарубежных компаний.

Проведён сравнительный анализ четырёх инструментов: Google Forms, Online Test Pad, Quizizz и Moodle. Ни одно из рассмотренных решений не охватывает весь необходимый набор функциональных и архитектурных характеристик. Сформированные функциональные и нефункциональные требования фиксируют целевые свойства разрабатываемой системы и служат основой для выбора технологического стека и проектирования архитектуры.

2 Проектирование архитектуры системы

2.1 Общая архитектура системы

Система реализована по трёхуровневой клиент-серверной архитектуре [9]: уровень представления в виде клиентского веб-приложения, уровень бизнес-логики в виде сервера приложений и уровень данных, состоящий из реляционной базы данных и объектного хранилища медиафайлов. Клиент управляет представлением и взаимодействием с пользователем, сервер — бизнес-логикой, хранением данных и управлением доступом.

Выбор монолитной архитектуры для серверной части основан на критериях масштаба проекта и состава команды. Монолитное приложение обеспечивает простоту развёртывания как единый артефакт сборки, атомарность транзакций в рамках единой базы данных и низкую операционную сложность — не требуется инфраструктура оркестрации контейнеров, распределённая трассировка или межсервисная коммуникация. Микросервисная архитектура для проекта с одним разработчиком и компактной предметной областью создаёт операционную сложность, несоизмеримую её преимуществам. При необходимости монолитная архитектура допускает горизонтальное масштабирование серверных экземпляров через балансировщик нагрузки.

При проектировании применяются принципы методологии двенадцати факторов [10]: зависимости объявляются явно и изолируются, конфигурация хранится в переменных окружения, окружения разработки и эксплуатации максимально идентичны за счёт контейнеризации.

2.1.1 Архитектура клиентской части

Клиентская часть реализуется как одностраничное веб-приложение, обеспечивающее отзывчивый интерфейс за счёт динамической перерисовки содержимого без полной перезагрузки страницы.

Для организации исходного кода применяется методология **Feature-Sliced Design** [11] — подход к архитектуре фронтенд-приложений, основанный на послойной декомпозиции с однонаправленными зависимостями. Ключевое преимущество методологии — строгое правило зависимостей: каждый слой может импортировать модули только из нижележащих слоёв, что исключает

циклические зависимости и делает структуру проекта предсказуемой по мере его роста. Методология не привязана к конкретным технологиям и применима к любому стеку для разработки пользовательских интерфейсов.

Архитектура клиентской части включает следующие слои от верхнего к нижнему:

- app — точка входа приложения: маршрутизация, глобальное состояние, инициализация системы аутентификации, провайдеры контекста.
- pages — маршрутные компоненты верхнего уровня, соответствующие URL-адресам системы: каталог тестов, редактор теста, прохождение теста, просмотр результатов, аутентификация. Страницы не содержат бизнес-логики — они komponуют виджеты и features.
- widgets — переиспользуемые составные блоки интерфейса: карточка теста в каталоге, редактор вопроса с прикреплением медиафайлов, компонент отображения вопроса при прохождении, панель разбора ответов, строка поиска с фильтрацией по тегам.
- features — изолированные сценарии взаимодействия пользователя с системой: аутентификация, управление тестами, прохождение теста и отправка ответов, поиск в публичном каталоге, рендеринг форматированного текста и формул, загрузка медиафайлов на сервер.
- entities — доменные объекты с управлением состоянием: тесты, вопросы, попытки прохождения, пользователи. Здесь описываются типы данных, нормализованное клиентское состояние и селекторы для доступа к нему.
- shared — переиспользуемая инфраструктура без доменного контекста: HTTP-клиент для взаимодействия с API, библиотека UI-примитивов, модуль рендеринга математических выражений, вспомогательные функции.

Каждый слой имеет чётко определённую зону ответственности. Верхние слои зависят от нижних, обратные зависимости запрещены. Модули внутри одного слоя изолированы друг от друга. Инфраструктурные компоненты из слоя shared доступны всем вышележащим слоям. Компонентная диаграмма клиентской части представлена на рисунке 5.

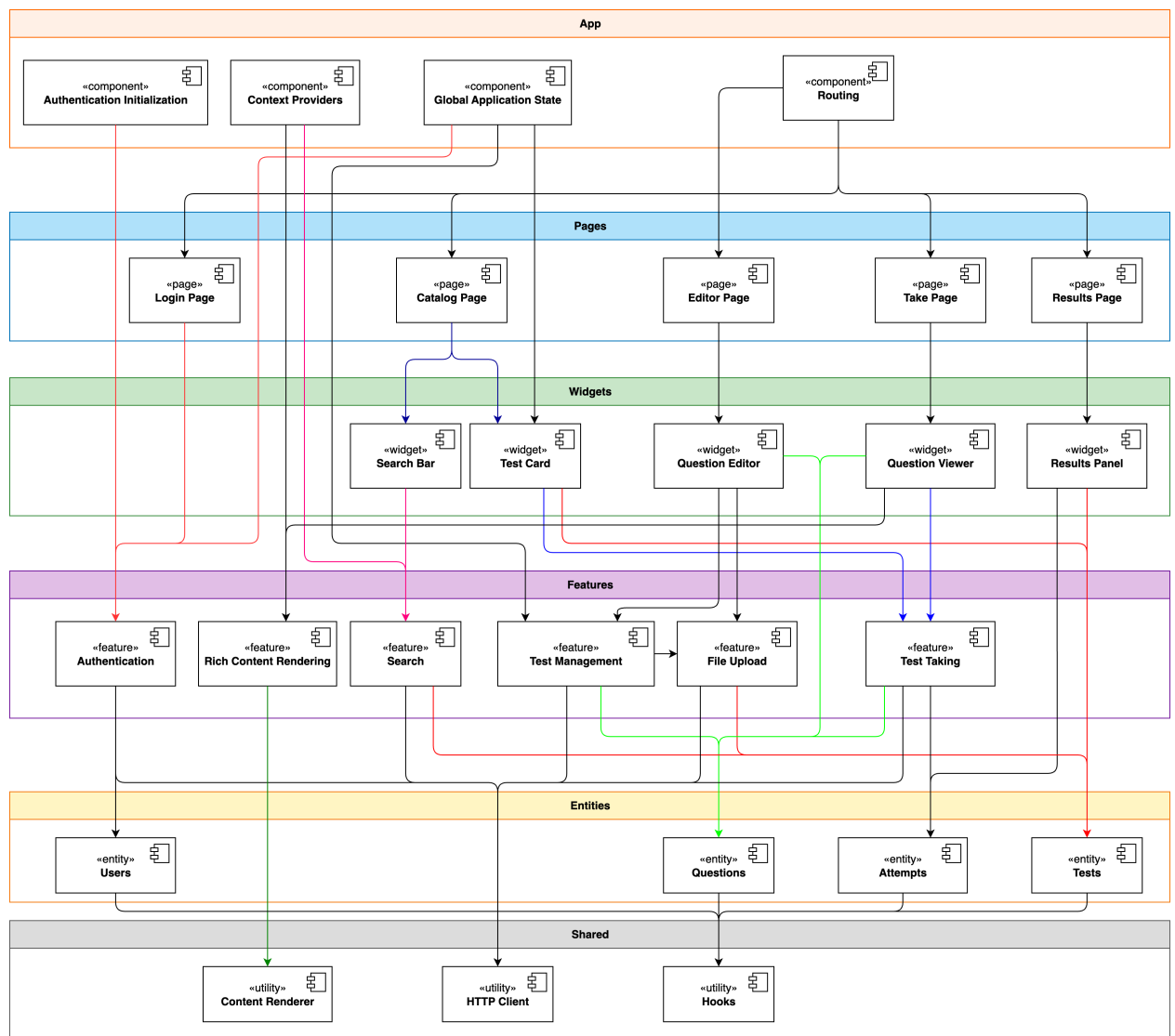


Рисунок 5 – Компонентная диаграмма клиентской части (Feature-Sliced Design)

2.1.2 Архитектура серверной части

Серверная часть строится на принципах **многослойной архитектуры** с однонаправленными зависимостями между слоями [5]. Каждый слой имеет строго ограниченную зону ответственности.

Слои серверной архитектуры от верхнего к нижнему:

- Слой представления — приём и обработка HTTP-запросов, декларативная маршрутизация, валидация входных данных, сериализация ответов, обработка ошибок протокола. Этот слой не содержит бизнес-логики — он делегирует вызовы слою приложения и преобразует доменные объекты в формат передачи данных.

- Слой приложения — реализация прикладных сценариев использования системы: аутентификация пользователей, управление

жизненным циклом тестов, проверка прав доступа, валидация и сохранение медиафайлов, обработка попыток прохождения тестов, поиск и фильтрация в каталоге.

– Слой доступа к данным — изоляция операций персистентности от бизнес-логики через паттерн **Repository**. Каждый репозиторий инкапсулирует доступ к одному агрегату и предоставляет методы для создания, чтения, обновления и удаления сущностей. Репозитории работают с сессией базы данных и скрывают детали SQL-запросов.

– Слой моделей — доменные сущности, отображающие структуру предметной области: пользователи, тесты, вопросы, варианты ответов, попытки прохождения. Модели описывают схему базы данных и связи между таблицами.

Поперечные слои обслуживают все вышестоящие слои: схемы передачи данных определяют контракты входящих и исходящих данных для HTTP API и обеспечивают валидацию на границе системы; модули конфигурации и безопасности управляют параметрами приложения, внедрением зависимостей и утилитами криптографии для хеширования паролей и подписи токенов аутентификации.

Основные архитектурные принципы серверной части: разделение ответственности — бизнес-логика изолирована от деталей протокола и персистентности; инверсия зависимостей — слой приложения зависит от абстракций репозиториев, а не от конкретной реализации доступа к данным; единый источник истины — доменные модели являются каноническим представлением структуры данных; управление транзакциями — атомарность операций обеспечивается на уровне репозиториев через сессии базы данных.

Для взаимодействия с объектным хранилищем медиафайлов применяется S3-совместимый API, инкапсулированный в слой приложения. Генерация подписанных URL для прямого доступа клиентов к медиаконтенту выполняется сервером без проксирования бинарных данных. Управление схемой базы данных осуществляется через систему миграций, обеспечивающую версионный контроль изменений структуры данных. Компонентная диаграмма серверной части представлена на рисунке 6.

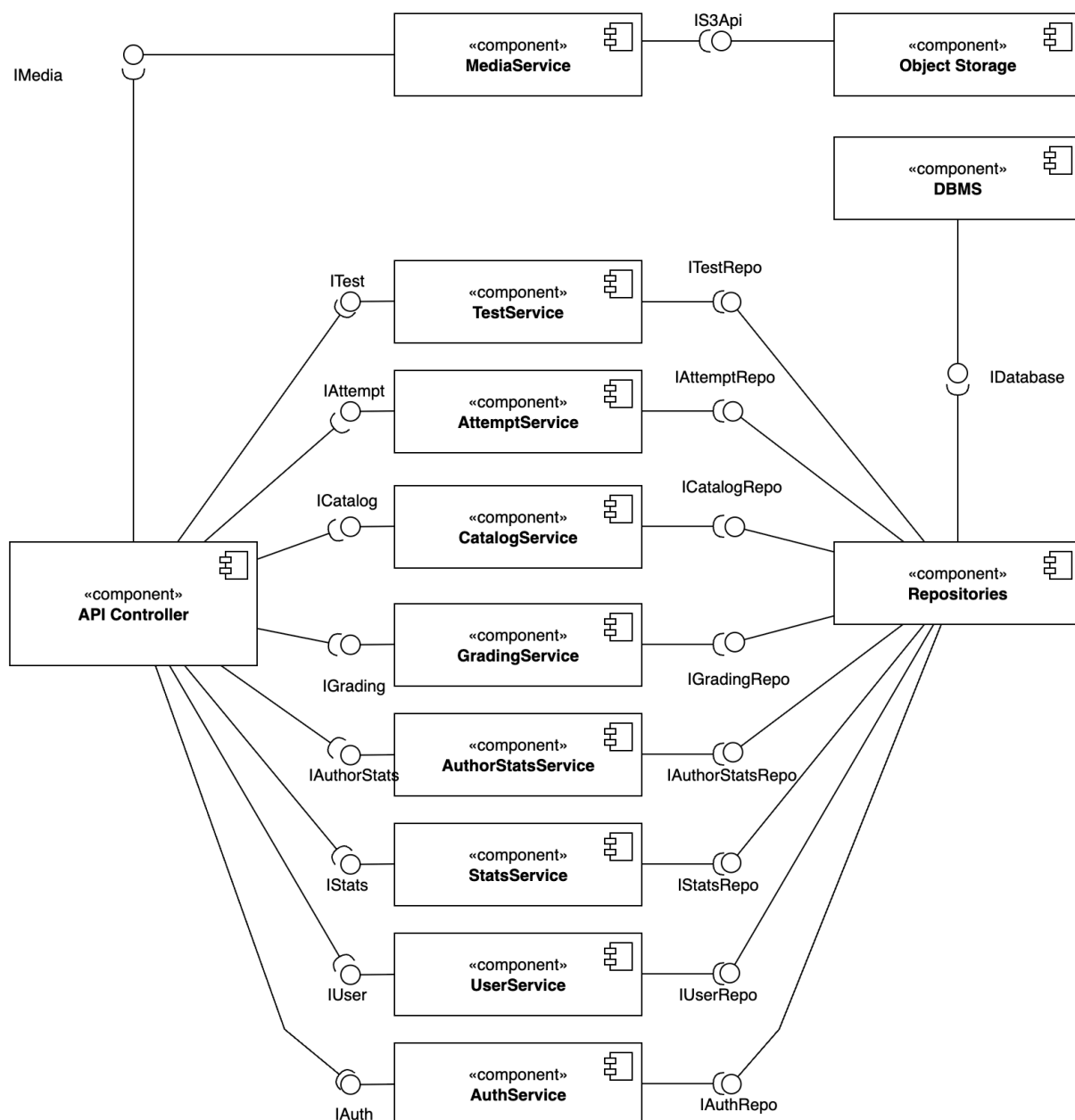


Рисунок 6 – Компонентная диаграмма серверной части (многослойная архитектура)

2.1.3 Модель данных

Модель данных спроектирована в соответствии с выявленными функциональными требованиями и представлена ER-диаграммой на рисунке 7. Применяется реляционная модель с нормализацией до третьей нормальной формы.

Центральными сущностями системы являются:

- User — учётная запись автора или участника тестирования. Содержит аутентификационные данные и профильную информацию.

– Test — образовательный тест, созданный автором. Связан отношением «многие к одному» с сущностью User. Включает метаданные: название, описание, режим доступа, временные метки создания и изменения.

– Question — элемент теста. Связан отношением «многие к одному» с сущностью Test. Содержит текст вопроса, тип ответа, ссылку на медиафайл, порядковый номер.

– Option — возможный вариант ответа для вопросов с выбором. Связан отношением «многие к одному» с сущностью Question. Хранится в отдельной таблице, что обеспечивает произвольное количество вариантов на вопрос. Включает атрибут корректности.

– Tag — метка для классификации тестов. Связан отношением «многие ко многим» с сущностью Test через ассоциативную таблицу. Обеспечивает категоризацию и поиск тестов без дублирования названий тегов.

– Attempt — фиксация прохождения теста пользователем. Связана отношением «многие к одному» с сущностями User и Test. Содержит временные метки начала и завершения, итоговый балл.

– Answer — ответ пользователя на конкретный вопрос в рамках попытки. Связан отношением «многие к одному» с сущностями Attempt и Question. Содержит текст ответа пользователя и результат проверки. Результат проверки рассчитывается сервером единожды при приёме ответа и сохраняется, исключая пересчёт при каждом запросе.

Особенностями проектирования модели данных являются следующие: ссылки на медиафайлы хранятся как URL, при этом бинарный контент не размещается в реляционной базе данных, а хранится в объектном хранилище; ассоциативная таблица для тегов реализует нормализованное отношение «многие ко многим», позволяя повторно использовать теги без избыточности; результаты проверки ответов денормализованы и хранятся совместно с ответами для минимизации вычислительной нагрузки при отображении результатов; порядок вопросов задаётся явным атрибутом порядкового номера, обеспечивая контроль последовательности при прохождении теста.

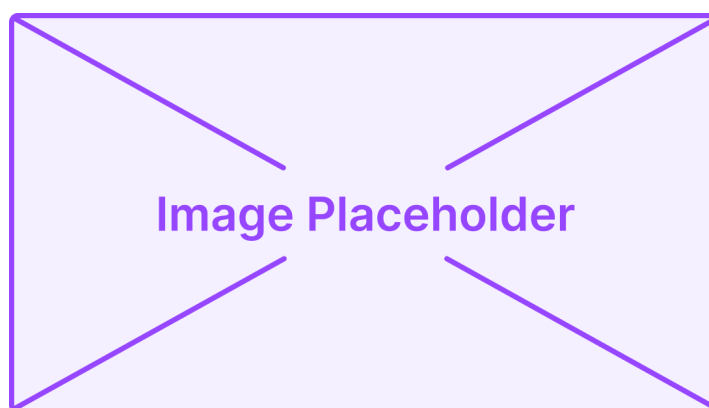


Рисунок 7 – ER-диаграмма базы данных системы

2.1.4 Модель развёртывания

Система проектируется для развёртывания на выделенном виртуальном сервере с применением контейнеризации. Архитектура развёртывания представлена на диаграмме на рисунке 8.

Инфраструктура системы включает следующие компоненты:

- Обратный прокси-сервер — точка входа для всех внешних запросов. Выполняет SSL-терминирование для шифрования трафика по протоколу HTTPS, маршрутизацию запросов между статическим контентом и API, сжатие ответов, кеширование статических ресурсов. Статические файлы клиентского приложения раздаются напрямую; запросы к API перенаправляются на сервер приложений.

- Сервер приложений — контейнер с серверной логикой системы. Обрабатывает HTTP-запросы к API, выполняет бизнес-логику, взаимодействует с базой данных и объектным хранилищем. Не имеет публичного доступа из внешней сети — принимает запросы исключительно от обратного прокси-сервера.

- Сервер базы данных — реляционная СУБД для хранения структурированных данных: пользователи, тесты, вопросы, попытки. Доступна только серверу приложений через внутреннюю сеть. Публичный доступ к порту базы данных закрыт для минимизации поверхности атаки.

– Объектное хранилище — служба хранения медиафайлов с S3-совместимым API. Сервер приложений загружает файлы и генерирует подписанные URL для прямого доступа клиентов. Клиентское приложение скачивает медиаконтент напрямую из хранилища, минуя сервер приложений и снижая нагрузку на API.

Все компоненты изолированы в контейнерах и взаимодействуют через внутреннюю виртуальную сеть. Обратный прокси-сервер является единственным компонентом с публичным сетевым интерфейсом. Сервер базы данных и объектное хранилище недоступны извне. Данные базы данных и объектного хранилища сохраняются в постоянных томах, обеспечивая сохранность информации при перезапуске контейнеров. Резервное копирование выполняется на уровне томов.

Архитектура допускает горизонтальное масштабирование серверов приложений: обратный прокси-сервер может распределять нагрузку между несколькими экземплярами API. База данных и объектное хранилище масштабируются вертикально или через репликацию. Требования к окружению включают минимальную версию Linux-ядра для поддержки контейнеризации, поддержку изоляции на уровне контейнеров и SSL-сертификат для шифрования трафика.

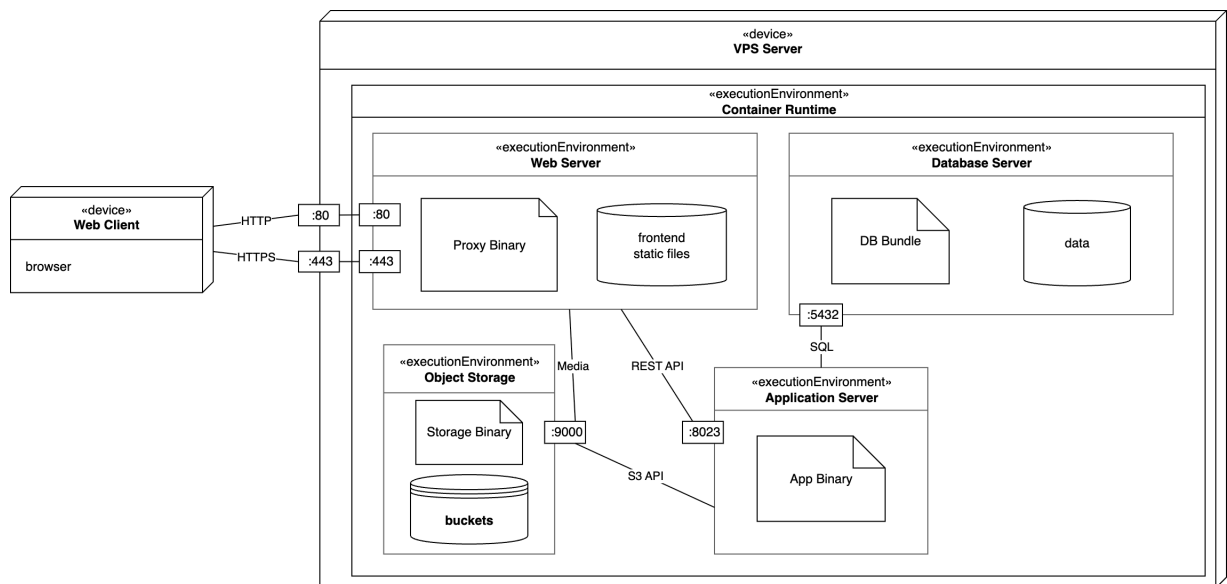


Рисунок 8 – Диаграмма развёртывания системы на VPS

2.2 Выбор технологического стека

2.2.1 Клиентская часть

Для реализации компонентного интерфейса требуется декларативный UI-фреймворк с широкой экосистемой. **React** [12] отвечает этим требованиям: компонентная модель с хуками обеспечивает переиспользование логики, а Virtual DOM оптимизирует перерисовки при частых обновлениях. Прохождение теста является динамичным сценарием с частой сменой состояния. В качестве языка выбран **TypeScript**, который устраняет целый класс ошибок на этапе компиляции, критично при сложной типизации ответов API. Инструмент сборки **Vite** обеспечивает быстрый HMR в разработке и эффективный бандл в продакшне благодаря нативным ES-модулям.

Серверное состояние требует кеширования, фоновой синхронизации и дедупликации запросов. TanStack Query решает эти задачи декларативно: запрос к API кешируется, обновляется при фокусировке окна и переиспользуется на всех страницах без дополнительного кода. Аналоги SWR и Apollo Client либо менее функциональны, либо ориентированы на GraphQL. Для локального клиентского состояния выбран Zustand — минималистичная

библиотека без бойлерплейта, которая не требует провайдеров и не создаёт лишних ре-рендеров.

Компонентная база строится на примитивах Radix UI с утилитарными стилями Tailwind CSS. Radix предоставляет доступные нестилизованные компоненты: диалоги, раскрывающиеся списки, переключатели. Это устраняет необходимость реализовывать поведение клавиатурной навигации вручную. Tailwind ускоряет написание стилей через именованные классы и исключает конфликты CSS-классов в монолитном приложении.

Отдельного внимания заслуживает выбор инструмента для отображения форматированного текста и математических формул в вопросах. Популярной альтернативой является KaTeX — быстрый рендерер LaTeX-формул для браузера. Однако KaTeX ограничен только математическими выражениями и требует от авторов знания LaTeX-синтаксиса, который сложнее для нематематиков. Вместо него выбран **typst.ts** [13] — WASM-компилятор языка Typst, запускающийся непосредственно в браузере. Typst поддерживает как математические формулы, так и произвольное форматирование текста: списки, жирный шрифт, цитаты, таблицы. Язык имеет более простой и читаемый синтаксис, а его официальная документация даёт исчерпывающие примеры. Авторы тестов получают единый инструмент для всего содержимого вопроса, а не отдельный рендерер для математики.

2.2.2 Серверная часть

Для реализации REST API выбран **Python** в качестве основного языка — он сочетает читаемый синтаксис, развитую асинхронную экосистему на базе `asyncio` и строгую систему аннотаций типов, которая улучшает статический анализ кода. Среди Python-фреймворков выбран **FastAPI** [14]. По данным опроса Stack Overflow Developer Survey 2025 [15], FastAPI занимает пятое место среди веб-фреймворков по популярности у профессиональных разработчиков с показателем 20,8%, опережая Express, Angular и ASP.NET Core. Распределение популярности фреймворков представлено на рисунке 9.

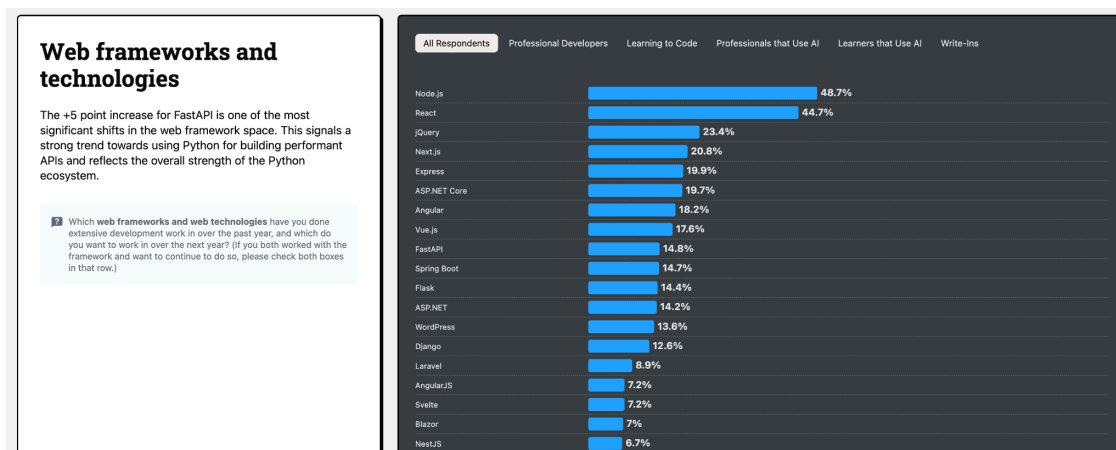


Рисунок 9 – Популярность веб-фреймворков среди профессиональных разработчиков (Stack Overflow Developer Survey 2025)

Ключевые преимущества FastAPI для данного проекта: нативная `asyncio`-модель выполнения — запросы не блокируют друг друга при ожидании ответа базы данных; встроенный `Dependency Injection` упрощает передачу сессии БД в репозитории; автоматическая генерация `OpenAPI`-документации из кода снижает рассинхронизацию между кодом и спецификацией; полная интеграция с `Pydantic` для валидации. `Django` и `Flask` менее удобны для чисто `API`-ориентированных приложений.

`SQLAlchemy` в `asyncio`-режиме [16] является персистентным движком системы. В режиме `AsyncSession` с драйвером `asyncpg` запросы к `PostgreSQL` не блокируют `event loop` `FastAPI`, что критично для производительности при конкурентных прохождении тестов. Управление миграциями через `Alembic` позволяет вносить изменения в схему базы данных под контролем версий без потери данных.

`Pydantic` обеспечивает декларативную валидацию входящих данных: схемы `DTO` описываются как обычные `Python`-классы с аннотациями, а `FastAPI` автоматически интегрирует их в маршруты. `Argon2id` выбран для хеширования паролей как алгоритм, устойчивый к `GPU`-атакам за счёт высокого потребления памяти. `JWT` реализует безсессионную аутентификацию: токен содержит идентификатор пользователя и срок действия, что исключает обращение к базе данных при каждом запросе.

2.2.3 Хранение данных

PostgreSQL выбран в качестве реляционной СУБД [17]. Критерии выбора: строгие ACID-транзакции для корректного завершения попытки при параллельной отправке ответов, поддержка enum-типов и JSONB для гибкого хранения метаданных при необходимости, богатый набор индексов для полнотекстового поиска по названиям тестов. Реляционная модель обоснована характером данных: между сущностями системы существуют строгие ссылочные ограничения. Документоориентированные СУБД не дают преимуществ при наличии фиксированной схемы и транзакций между несколькими коллекциями.

MinIO [18] — S3-совместимое объектное хранилище с открытым исходным кодом, разворачиваемое как Docker-контейнер. Выбор обусловлен self-hosted развёртыванием: данные не покидают собственный сервер. Хранилище совместимо с Amazon S3 API, что позволяет использовать один клиент для работы с любым S3-совместимым хранилищем. Поддержка presigned URL позволяет клиенту загружать и скачивать файлы напрямую, минуя API. Встроенный веб-интерфейс упрощает администрирование. AWS S3 как альтернатива требует внешнего провайдера и порождает зависимость от зарубежной инфраструктуры.

2.2.4 Инфраструктура

Из диаграммы развёртывания следует, что система требует воспроизводимой упаковки пяти компонентов и их оркестрации на одном хосте. **Docker Compose** [19] решает эту задачу декларативно: конфигурация одним файлом описывает образы, переменные окружения, сетевую изоляцию и volumes для персистентности данных. Полный разворот с нуля занимает одну команду. Альтернативой является Kubernetes, однако для одного VPS и одного разработчика его операционная сложность является избыточной.

Nginx выполняет роль обратного прокси-сервера и раздаёт статику фронтенда. Статические файлы Vite-бандла хранятся в отдельном volume и монтируются в nginx, что позволяет обновлять фронтенд без перезапуска API. Конфигурация nginx покрывает SSL-терминирование, проксирование API-запросов и добавление заголовков безопасности.

2.3 Итоги

В данной главе спроектирована архитектура системы интерактивного конструктора образовательных тестов и определён технологический стек для её реализации.

Выбрана клиент-серверная трёхуровневая монолитная архитектура, обеспечивающая централизованное хранение данных, безопасность проверки результатов и простоту развёртывания. Клиентская часть спроектирована как одностраничное веб-приложение с применением методологии Feature-Sliced Design: шесть слоёв с однонаправленными зависимостями. Серверная часть организована по принципу многослойной архитектуры с четырьмя слоями и применением паттерна Repository для изоляции персистентности. Модель данных спроектирована в третьей нормальной форме и включает восемь сущностей с денормализованным хранением результатов проверки для оптимизации производительности. Модель развёртывания предполагает контейнеризованную архитектуру на выделенном виртуальном сервере с изоляцией компонентов и закрытым доступом к базе данных.

Определён технологический стек системы. Клиентская часть реализуется на React с TypeScript, организована по методологии Feature-Sliced Design, использует TanStack Query для управления серверным состоянием, Zustand для локального состояния, Radix UI с Tailwind CSS для компонентной базы и typst.ts для рендеринга математических формул и форматированного текста. Серверная часть реализуется на Python с фреймворком FastAPI, который занимает пятое место по популярности среди веб-фреймворков по данным Stack Overflow Developer Survey 2025. Для работы с базой данных применяется SQLAlchemy в asyncio-режиме с системой миграций Alembic. Хранение данных организовано на PostgreSQL для структурированных данных и MinIO для медиафайлов. Развёртывание осуществляется через Docker Compose с Nginx в качестве обратного прокси-сервера.

Спроектированная архитектура и выбранный технологический стек соответствуют функциональным и нефункциональным требованиям, сформулированным в первой главе, и обеспечивают основу для реализации системы.

3 Разработка, тестирование и развертывание системы

3.1 Разработка серверной части

Серверная часть реализована на языке Python с использованием фреймворка FastAPI. Код организован по принципам многослойной архитектуры: слой представления (API-роутеры), слой приложения (сервисы), слой доступа к данным (репозитории) и слой моделей (SQLAlchemy ORM). Такое разделение позволяет изолировать бизнес-логику от деталей HTTP-протокола и способа хранения данных.

Точкой входа приложения является файл `app/main.py`, в котором создаётся экземпляр FastAPI, регистрируются маршруты, middleware и обработчики исключений. Жизненный цикл приложения управляется через контекстный менеджер `lifespan`, инициализирующий подключение к объектному хранилищу при старте.

Листинг 1 – Инициализация приложения FastAPI и подключение роутера

```
app = fastapi.FastAPI(
    title=settings.APP_NAME,
    version=settings.APP_VERSION,
    docs_url="/api/docs",
    openapi_url="/api/openapi.json",
    lifespan=lifespan,
)
app.include_router(api_v1.api_router, prefix=settings.API_V1_PREFIX)
```

Маршрутизация оформлена в виде подключаемых роутеров (APIRouter). Каждый роутер отвечает за отдельную предметную область: аутентификация, управление тестами, прохождение тестов, каталог. Пример объявления endpoint'а создания теста представлен в листинге 2.

Листинг 2 – Endpoint создания теста с внедрением зависимостей

```
@router.post("/", response_model=TestResponse, status_code=201)
async def create_test(
    data: TestCreate,
    current_user: User = Depends(get_current_user),
    svc: TestService = Depends(_get_test_service),
) -> TestResponse:
    test = await svc.create_test(author_id=current_user.id, data=data)
    return _map_test_response(test)
```

Бизнес-логика инкапсулирована в сервисах. Сервис `TestService` координирует создание теста, связывание тегов и валидацию прав доступа. Репозитории скрывают детали SQL-запросов; сервисы работают с абстракциями `AsyncSession` и не зависят от конкретной СУБД. Валидация входящих данных на границе системы выполняется декларативно с помощью Pydantic [20]: схемы DTO описываются как Python-классы с аннотациями типов, а FastAPI автоматически интегрирует их в маршруты.

Листинг 3 – Сервисный слой: создание теста с привязкой тегов

```
class TestService:
    def __init__(self, db: AsyncSession):
        self.db = db
        self.test_repo = TestRepository(db)
        self.tag_repo = TagRepository(db)

    async def create_test(self, author_id: int, data: TestCreate) -> Test:
        test = await self.test_repo.create({"author_id": author_id, ...})
        if data.tag_names:
            tag_ids = [(await self.tag_repo.get_or_create(n)).id for n in
data.tag_names]
            await self.test_repo.add_tags(test.id, tag_ids)
        return test
```

Управление схемой базы данных выполняется через Alembic [21]. Миграции описывают изменения структуры таблиц и связей; их применение осуществляется командой `alembic upgrade head` перед запуском приложения. Это обеспечивает воспроизводимость схемы БД в любой среде.

3.2 Разработка клиентской части

Клиентская часть реализована как одностраничное приложение на React с TypeScript. Сборка производится Vite; development-сервер запускается командой `vite`, продакшн-бандл — `vite build`. Архитектура следует методологии Feature-Sliced Design: модули распределены по слоям `app`, `pages`, `widgets`, `features`, `entities`, `shared`.

Серверное состояние (тесты, попытки, результаты) запрашивается через TanStack Query, что обеспечивает кеширование, фоновое обновление и дедупликацию запросов. Локальное состояние интерфейса управляется

Zustand. Компонентная база построена на примитивах Radix UI со стилизацией Tailwind CSS.

Рендеринг математических формул и форматированного текста выполняется с помощью WASM-компилятора `typst.ts`, запускающегося непосредственно в браузере. Это позволяет авторам тестов использовать единый язык разметки для текстов, списков, таблиц и математических выражений.

3.3 Тестирование системы

3.3.1 Модульное и интеграционное тестирование

Серверная часть покрыта тестами на базе `pytest`. Тесты запускаются в изолированной схеме PostgreSQL, которая создаётся перед каждым тестовым модулем и удаляется после. Фикстура `http_client` предоставляет асинхронный HTTP-клиент, обращающийся к запущенному в тестовом режиме приложению FastAPI. Проверяются сценарии аутентификации, CRUD-операции над тестами, прохождение тестов и выставление оценок.

3.3.2 Фаззинг тестирование

В соответствии с требованием о проверке устойчивости API к некорректным входным данным выполнено фаззинг-тестирование с применением библиотеки `Hypothesis`. Цель фаззинга — убедиться, что API никогда не возвращает HTTP 500 при любых входных данных: ожидаемые ответы на невалидный ввод — 400, 401, 403, 404, 409, 422.

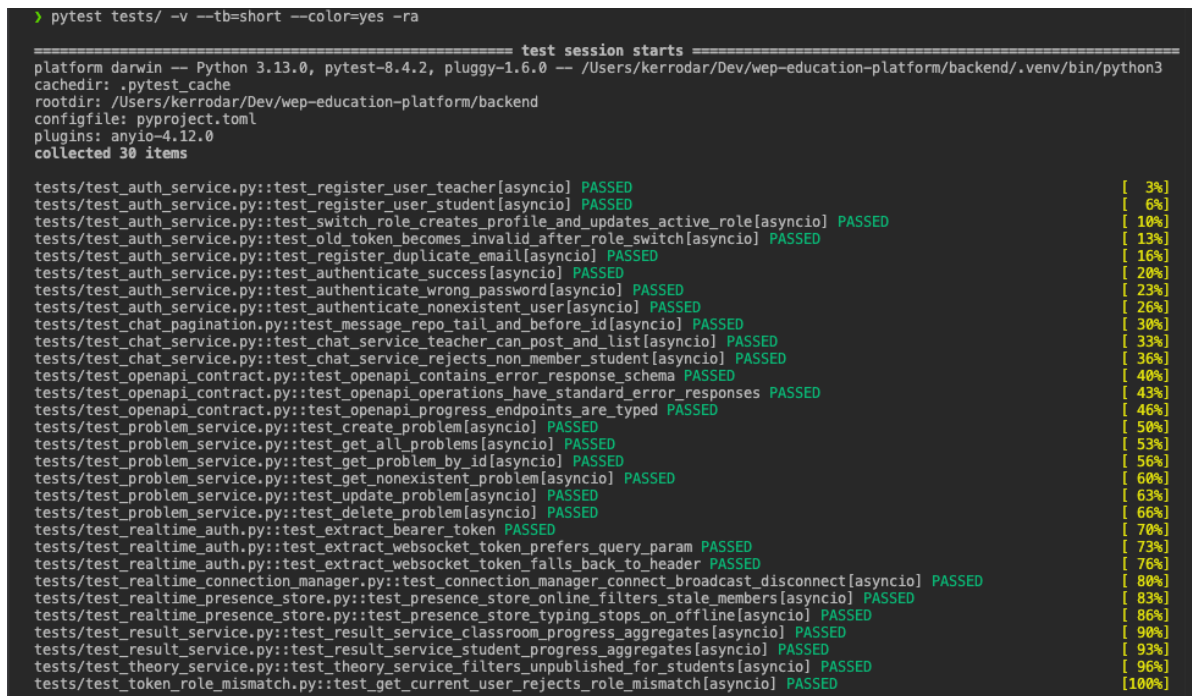
Фаззинг покрывает endpoint'ы регистрации, входа, создания тестов, получения теста по идентификатору, каталога, начала попытки и отправки ответов. Генерируются случайные строки, целые числа, булевы значения и структурированные объекты в граничных диапазонах. Отдельно проверяются вредоносные payload'ы: SQL-инъекции и XSS. Листинг 4 демонстрирует фрагмент fuzz-теста.

Листинг 4 – Фаззинг-тест создания теста с генерацией случайных данных

```
NEVER_500 = {200, 201, 400, 401, 403, 404, 409, 422}

@given(
    title=st.text(min_size=0, max_size=512),
    description=st.one_of(st.none(), st.text(max_size=4096)),
    is_public=st.booleans(),
    time_limit=st.one_of(st.none(), st.integers(min_value=-100,
max_value=1_000_000)),
)
@settings(max_examples=30, suppress_health_check=_SUPPRESS, deadline=None)
async def test_fuzz_create_test(http_client, title, description, is_public,
time_limit):
    token = await _register_and_login(http_client)
    resp = await http_client.post("/api/v1/tests/", json={
        "title": title, "description": description,
        "is_public": is_public, "time_limit_minutes": time_limit,
    }, headers={"Authorization": f"Bearer {token}"})
    assert resp.status_code in NEVER_500, f"500 on create_test: {resp.text}"
```

Результаты тестирования (запуск полного набора pytest) представлены на рисунке 10.



```
> pytest tests/ -v --tb=short --color=yes -ra

===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.4.2, pluggy-1.6.0 -- /Users/kerrodar/Dev/wep-education-platform/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/kerrodar/Dev/wep-education-platform/backend
configfile: pyproject.toml
plugins: anyio-4.12.0
collected 30 items

tests/test_auth_service.py::test_register_user_teacher[asyncio] PASSED [ 3%]
tests/test_auth_service.py::test_register_user_student[asyncio] PASSED [ 6%]
tests/test_auth_service.py::test_switch_role_creates_profile_and_updates_active_role[asyncio] PASSED [ 10%]
tests/test_auth_service.py::test_old_token_becomes_invalid_after_role_switch[asyncio] PASSED [ 13%]
tests/test_auth_service.py::test_register_duplicate_email[asyncio] PASSED [ 16%]
tests/test_auth_service.py::test_authenticate_success[asyncio] PASSED [ 20%]
tests/test_auth_service.py::test_authenticate_wrong_password[asyncio] PASSED [ 23%]
tests/test_auth_service.py::test_authenticate_nonexistent_user[asyncio] PASSED [ 26%]
tests/test_chat_pagination.py::test_message_repo_tail_and_before_id[asyncio] PASSED [ 30%]
tests/test_chat_service.py::test_chat_service_teacher_can_post_and_list[asyncio] PASSED [ 33%]
tests/test_chat_service.py::test_chat_service_rejects_non_member_student[asyncio] PASSED [ 36%]
tests/test_openapi_contract.py::test_openapi_contract_contains_error_response_schema PASSED [ 40%]
tests/test_openapi_contract.py::test_openapi_operations_have_standard_error_responses PASSED [ 43%]
tests/test_openapi_contract.py::test_openapi_progress_endpoints_are_typed PASSED [ 46%]
tests/test_problem_service.py::test_create_problem[asyncio] PASSED [ 50%]
tests/test_problem_service.py::test_get_all_problems[asyncio] PASSED [ 53%]
tests/test_problem_service.py::test_get_problem_by_id[asyncio] PASSED [ 56%]
tests/test_problem_service.py::test_get_nonexistent_problem[asyncio] PASSED [ 60%]
tests/test_problem_service.py::test_update_problem[asyncio] PASSED [ 63%]
tests/test_problem_service.py::test_delete_problem[asyncio] PASSED [ 66%]
tests/test_realtime_auth.py::test_extract_bearer_token PASSED [ 70%]
tests/test_realtime_auth.py::test_extract_websocket_token_prefers_query_param PASSED [ 73%]
tests/test_realtime_auth.py::test_extract_websocket_token_falls_back_to_header PASSED [ 76%]
tests/test_realtime_connection_manager.py::test_connection_manager_connect_broadcast_disconnect[asyncio] PASSED [ 80%]
tests/test_realtime_presence_store.py::test_presence_store_online_filters_stale_members[asyncio] PASSED [ 83%]
tests/test_realtime_presence_store.py::test_presence_store_typing_stops_on_offline[asyncio] PASSED [ 86%]
tests/test_result_service.py::test_result_service_classroom_progress_aggregates[asyncio] PASSED [ 90%]
tests/test_result_service.py::test_result_service_student_progress_aggregates[asyncio] PASSED [ 93%]
tests/test_theory_service.py::test_theory_service_filters_unpublished_for_students[asyncio] PASSED [ 96%]
tests/test_token_role_mismatch.py::test_get_current_user_rejects_role_mismatch[asyncio] PASSED [100%]
```

Рисунок 10 – Результаты выполнения backend-тестов: модульные, интеграционные и fuzz

Все тесты пройдены без единого внутреннего серверного сбоя (HTTP 500). Граничные значения корректно отклоняются валидацией Pydantic с кодом

422. Вредоносные payload'ы обрабатываются безопасно: параметризованные запросы SQLAlchemy исключают SQL-инъекции, а XSS-векторы сохраняются как обычный текст и не интерпретируются при выводе [22].

3.4 Развёртывание системы

3.4.1 Контейнеризация серверной части

Серверное приложение упаковывается в Docker-образ на базе `python:3.12-slim`. Инструмент `uv` используется для установки зависимостей; итоговый образ содержит только необходимые системные библиотеки (`libpq-dev`, `gcc`) и код приложения. Образ экспонирует порт 8023 и запускает `Uvicorn`. Листинг 5 – Dockerfile серверной части: сборка образа на Python 3.12 + `uv`

```
FROM python:3.12-slim
WORKDIR /app
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /usr/local/bin/
RUN apt-get update && apt-get install -y --no-install-recommends libpq-dev gcc \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt ./
RUN uv pip install --system --no-cache -r requirements.txt
COPY . .
EXPOSE 8023
CMD ["uv", "run", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8023"]
```

3.4.2 Контейнеризация клиентской части

Клиентская часть собирается в два этапа. Первый этап (`builder`) на базе `oven/bun:1-alpine` устанавливает зависимости и выполняет `production`-сборку `Vite`. Второй этап копирует статические файлы из `dist` в образ `nginx:alpine` и использует кастомный конфигурационный файл `nginx-prod.conf` для маршрутизации запросов.

3.4.3 Оркестрация production-среды

`Production`-развёртывание описывается файлом `docker-compose.prod.yml` [19] и включает четыре сервиса: обратный прокси `nginx` (порт 80/443), сервер приложений (порт 8023), `PostgreSQL` и `MinIO`. Все сервисы изолированы в общей виртуальной сети; база данных и объектное хранилище недоступны извне. Фронтенд зависит от состояния здоровья `backend` через `depends_on` с `condition: service_healthy`.

Листинг 6 – Фрагмент docker-compose.prod.yml: зависимости и проверки здоровья сервисов

```
services:
  frontend:
    build:
      context: ../frontend
      dockerfile: Dockerfile.prod
    ports: ["80:80", "443:443"]
    depends_on:
      backend: { condition: service_healthy }
  backend:
    build:
      context: ../backend
      dockerfile: Dockerfile
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}
@postgres:5432/${POSTGRES_DB}
    depends_on:
      postgres: { condition: service_healthy }
      minio: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8023/api/v1/health"]
      interval: 30s
      retries: 3
```

Переменные окружения (SECRET_KEY, POSTGRES_USER, MINIO_ACCESS_KEY и др.) хранятся в файле .env и не включаются в образы. SSL-сертификаты Let's Encrypt монтируются в nginx как read-only volume. Данные PostgreSQL и MinIO персистируются в именованных Docker-томах, что обеспечивает сохранность информации при перезапуске контейнеров.

3.5 Итоги

В данной главе описаны процессы разработки, тестирования и развёртывания системы. Серверная часть реализована на FastAPI с многослойной архитектурой и управлением схемой БД через Alembic; клиентская — на React с Feature-Sliced Design и рендерингом формул через typst.ts. Проведено фаззинг-тестирование API с использованием Hypothesis; установлено отсутствие ошибок 500 при обработке произвольных и вредоносных входных данных. Развёртывание производится

контейнеризованной средой Docker Compose с изоляцией компонентов и персистентными томами для данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана серверная часть веб-приложения — интерактивного конструктора образовательных тестов.

На первом этапе выполнен анализ предметной области систем автоматизированного тестирования. Описан сложившийся процесс создания и проведения тестирования в образовательных организациях, выявлены системные проблемы: отсутствие единого репозитория тестов, ограниченная поддержка мультимедиа и математических формул, трудности с управлением доступом, риски, связанные с хранением данных на серверах иностранных компаний. Проведён обзор четырёх широко используемых инструментов — Google Forms, Online Test Pad, Quizizz и Moodle — и построены матрицы функционального и архитектурного сравнения, показавшие нишу для разрабатываемой системы. Сформированы функциональные и нефункциональные требования, фиксирующие целевые свойства конструктора.

На втором этапе спроектирована архитектура системы. Выбрана клиент-серверная трёхуровневая монолитная архитектура. Клиентская часть спроектирована как одностраничное веб-приложение с применением методологии Feature-Sliced Design; серверная часть организована по принципу многослойной архитектуры с четырьмя слоями (представления, приложения, доступа к данным, моделей) и применением паттерна Repository. Спроектирована реляционная модель данных в третьей нормальной форме, включающая восемь сущностей. Определён технологический стек: Python + FastAPI, PostgreSQL, SQLAlchemy, Alembic, MinIO, React + TypeScript, Vite, TanStack Query, Zustand, typst.ts, Docker Compose.

На третьем этапе реализованы серверная и клиентская части приложения. Серверная часть включает REST API с автоматической валидацией Pydantic, сервисный слой с автоматической проверкой ответов, слой безопасности (Argon2id, JWT) и миграции Alembic. Клиентская часть реализована на React 18 + TypeScript с рендерингом математических формул и форматированного текста через WASM-компилятор typst.ts. Корректность реализации подтверждена набором автоматизированных тестов

(pytest), включая модульные, интеграционные и фаззинг-тесты с библиотекой Hypothesis. Фаззинг установил отсутствие внутренних серверных сбоев (HTTP 500) при обработке произвольных и вредоносных входных данных, включая SQL-инъекции и XSS. Развёртывание системы осуществляется через Docker Compose с Nginx в качестве обратного прокси-сервера и изолированными контейнерами для базы данных, объектного хранилища и сервера приложений.

Полученные результаты подтверждают применимость многослойной архитектуры и выбранного технологического стека для построения серверной части конструктора образовательных тестов. Разработанная архитектура допускает дальнейшее расширение функциональности и масштабирование без переписывания ядра системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Платформы дистанционного обучения в России 2025: рейтинг, сравнение и как выбрать [Электронный ресурс]. Москва: Diasoft, 2025. URL: <https://www.diasoft.ru/about/publications/21856/> (дата обращения: 16.05.2026).
2. Анна В.Г., Екатерина И.Г. Готовность педагога к созданию и использованию интерактивных учебных материалов в профессиональной деятельности: ISSN 1994–637 // Вестник Череповецкого государственного университета. 2023. № 2. Сс. 159–169.
3. Chang C.-K., Chien L.-C.-T. Enhancing Academic Performance with Generative AI-Based Quiz Platform: DOI: 10.1109/ICALT61570.2024.00062. 2024. Сс. 193–195.
4. Online Quiz Platform Market Size, Share, Trends & Forecast [Электронный ресурс]. Verified Market Research, 2024. URL: <https://www.verifiedmarketresearch.com/product/online-quiz-platform-market/> (дата обращения: 16.05.2026).
5. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения: ISBN 978–5–4461–1903–1. 2-е изд. Санкт-Петербург: Питер, 2021. 352 с.
6. Online Test Pad — онлайн-конструктор тестов, опросов, кроссвордов [Электронный ресурс]. URL: <https://onlinetestpad.com/> (дата обращения: 16.05.2026).
7. Promoting Formative Assessment with Quizizz: A Classroom Action Research Study [Электронный ресурс]. ResearchGate, 2024. URL: <https://www.researchgate.net/publication/379901904> (дата обращения: 16.05.2026).
8. Jimenez-Pitre I., Fernández J.L., Jiménez Pitre N. Usability Problems of LMS Learning Management Systems, Based on the Moodle Platform // Russian Law Journal. 2023. Т. 11, № 4.
9. Водяхо А.И. и др. Архитектурные решения информационных систем [Электронный ресурс]: учебник для вузов ISBN 978–5–507–44710–7. 3-е изд. Санкт-Петербург: Лань, 2022. 356 с. URL: <https://e.lanbook.com/book/254624> (дата обращения: 11.05.2026).

10. Wiggins A. The Twelve-Factor App [Электронный ресурс]. 2011. URL: <https://12factor.net/> (дата обращения: 16.05.2026).
11. Feature-Sliced Design: официальная документация [Электронный ресурс]. URL: <https://feature-sliced.github.io/documentation/ru/> (дата обращения: 11.05.2026).
12. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 11.05.2026).
13. typst.ts — Run Typst in JavaScript [Электронный ресурс]. URL: <https://github.com/Myriad-Dreamin/typst.ts> (дата обращения: 16.05.2026).
14. Ramírez S. FastAPI – официальная документация [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 11.05.2026).
15. Stack Overflow Developer Survey 2025 [Электронный ресурс]. 2025. URL: <https://survey.stackoverflow.co/2025/> (дата обращения: 11.05.2026).
16. SQLAlchemy 2.0 Documentation [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 11.05.2026).
17. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 11.05.2026).
18. MinIO Object Storage Documentation [Электронный ресурс]. MinIO, Inc. URL: <https://min.io/docs/> (дата обращения: 16.05.2026).
19. Docker Compose documentation [Электронный ресурс]. URL: <https://docs.docker.com/compose/> (дата обращения: 11.05.2026).
20. Pydantic V2 Documentation [Электронный ресурс]. URL: <https://docs.pydantic.dev/latest/> (дата обращения: 11.05.2026).
21. Alembic – официальная документация [Электронный ресурс]. URL: <https://alembic.sqlalchemy.org/> (дата обращения: 11.05.2026).
22. Хоффман Э. Безопасность веб-приложений: ISBN 978–5–4461–1706–8. Санкт-Петербург: Питер, 2021. 336 с.